Avrupa Birliği tarafından finanse edilmektedir

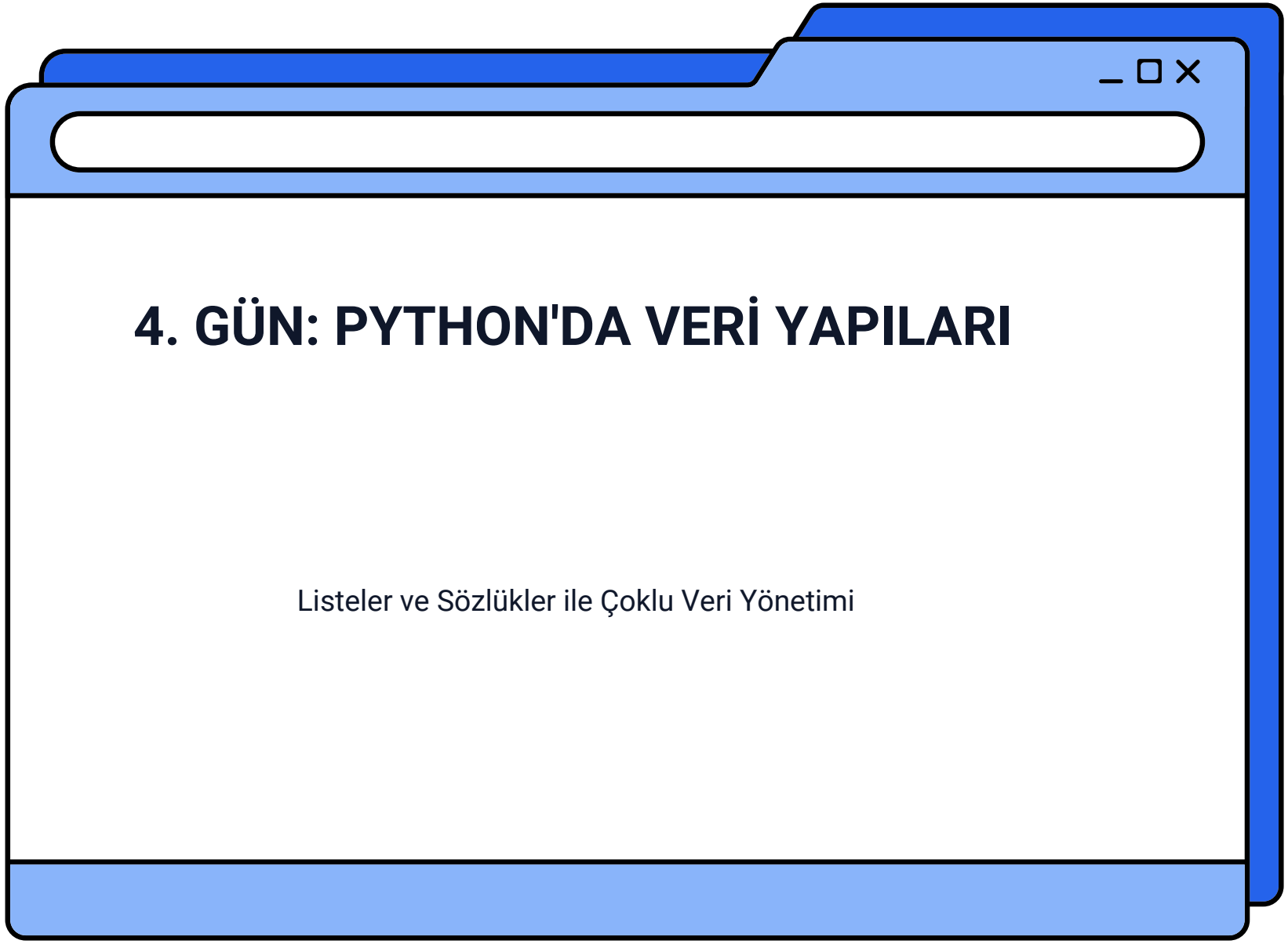
BUGÜNÜN Gençleri GELECEĞİN Meslekleri

UNDP
KARABÜK DİGEM

YAZILIM GELİŞTİRME EĞİTİMİ (SEVİYE 2)

GÜN 4

gokcedev.com
gokcesoylu24@gmail.com



4. GÜN: PYTHON'DA VERİ YAPILARI

Listeler ve Sözlükler ile Çoklu Veri Yönetimi

Tekil Veri Tipleri Hatırlatması ve Çoklu Veri İhtiyacı

Tekil Değişkenler:Şimdiye kadarherdeğişkende tek birdeğertuttuk:

- str: Metinsel veriler (ad = 'Ahmet')
- int: Tam sayılar (yas = 25)
- float: Ondalıklı sayılar (ort = 85.5)
- bool: Mantıksal değerler (aktif = True)

PYTHON KODU

```
# Tekil değişkenler (Zor ve ölçeklenemez yönetim) ogrenci1_not = 85
ogrenci2_not = 90
ogrenci3_not = 78
# Çoklu veri yapısı (Toplu ve pratik yönetim)
notlar = [85, 90, 78]
print("Not Sayısı:", len(notlar))
print("Tüm Notlar:", notlar)
```

Ekran Çıktısı:

```
Not Sayısı: 3
Tüm Notlar: [85, 90, 78]
```

Çoklu Veri Tutma İhtiyacı: 100 öğrencinin notunu saklamak için 100 ayrı değişken yazmak imkansızdır. Birden fazla veriyi tek bir yapıda toplamak için **Veri Yapıları** kullanılır.

Listeler (Lists) Nedir?

Python'dalisteler[ ]köşeliparantezlerkullanılarak oluşturulur. Birden fazla veriyi sıralı olarak depolar. Farklı veri tiplerini aynı listede barındırabilir.

PYTHON KODU

```
# Metin Listesi
meyveler = ["Elma", "Muz", "Çilek"]

# Sayı Listesi
sayilar = [10, 20, 30, 40]


# Karma Liste (Farklı Veri Tipleri Bir Arada)
ogrenci_bilgi = ["Ahmet Yılmaz", 18, 88.5, True]


print("Meyveler:", meyveler)
print("Sayılar:", sayilar)
print("Öğrenci Bilgisi:", ogrenci_bilgi)
```

Ekran Çıktısı:
Meyveler: ['Elma', 'Muz', 'Çilek']
Sayılar: [10, 20, 30, 40]
Öğrenci Bilgisi: ['Ahmet Yılmaz', 18, 88.5, True]

İndeks Mantığı ve Eleman Erişimi

Listeelemanlarınaerişmekiçin**0'danbaşlayan**indeksnumaraları kullanılır. Son elemana [-1] ile erişilir. len() toplam eleman sayısını verir.

PYTHON KODU

```
renkler = ["Kırmızı", "Yeşil", "Mavi", "Sarı"]


# Elemanlara İndeksle Erişim
ilk_eleman = renkler[0]
son_eleman = renkler[-1]

 # Eleman Güncelleme
 renkler[1] = "Turuncu" # "Yeşil" yerine "Turuncu" atandı


 print("İlk Eleman (0. indeks):", ilk_eleman)
 print("Son Eleman (-1. indeks):", son_eleman)
 print("Güncellenmiş Liste:", renkler)
 print("Toplam Eleman Sayısı:", len(renkler))
```

Ekran Çıktısı:
İlk Eleman (0. indeks): Kırmızı
Son Eleman (-1. indeks): Sarı
Güncellenmiş Liste: ['Kırmızı', 'Turuncu', 'Mavi', 'Sarı']
Toplam Eleman Sayısı: 4

GÖREV SLAYTI

ŞİMDİ SEN KODLA 1 – Oyun Envanteri Deneyi

GÖREV: Oyun Envanteri ve IndexError Deneyi

• Kendi bilgisayarınızda 5 elemanlı bir oyun envanteri listesi oluşturun: envanter = ['Kılıç', 'Kalkan', 'İksir', 'Kask', 'Ayakkabı'] • 1. İlk elemanı (0. indeks) ve son elemanı ([-1]) ekrana yazdırın.
• 2. Listede var olmayan 5. indekse erişmeyi deneyin: print(envanter[5])

PYTHON KODU

```
# Sen de Kendi Bilgisayarında Dene!
envanter = ["Kılıç", "Kalkan", "İksir", "Kask", "Ayakkabı"]

# 1. İlk ve Son Elemanı Yazdır print("İlk Eşya:", envanter[0])
print("Son Eşya:", envanter[-1])

# 2. Olmayan İndekse Erişim Deneyi (Hata Alacaksın!):
# print(envanter[5])
```

```
# Ekran Çıktısı:
İlk Eşya: Kılıç
Son Eşya: Ayakkabı
(envanter[5] çağrılırsa -> IndexError: list index out of range hatası oluşur!)
```

ÇÖZÜM SLAYTI

ŞİMDİ SEN KODLA 1 – Çözüm ve Hata Analizi

ÇÖZÜM AÇIKLAMASI:
- 5 elemanlı bir listenin indeksleri **0, 1, 2, 3, 4**'tür.
- envanter[5] ifadesi 6. elemanı aradığı için Python sınırların dışına çıkıldığını belirten IndexError hatası fırlatır.

PYTHON KODU

```
# TAM ÇALIŞAN ÇÖZÜM KODU
envanter = ["Kılıç", "Kalkan", "İksir", "Kask", "Ayakkabı"]



print("Envanterdeki Toplam Eşya:", len(envanter))
print("İlk Eleman (0. indeks):", envanter[0])
print("Son Eleman (-1. indeks):", envanter[-1])
```

```
print("--- İndeks Sınır Deneyi ---")
# 5 elemanlı listede son indeks 4'tür. 5 yazarsak HATA verir:
# print(envanter[5]) -> IndexError!
print("Doğru Son Eleman İndeksi (len - 1):", envanter[len(envanter)-1])
```

Ekran Çıktısı:

```
Envanterdeki Toplam Eşya: 5
İlk Eleman (0. indeks): Kılıç
Son Eleman (-1. indeks): Ayakkabı
--- İndeks Sınır Deneyi ---
Doğru Son Eleman İndeksi (len - 1): Ayakkabı
```

Temel Liste Metotları

Listeleri dinamik olarak değiştirmek için yerleşik metotlar kullanılır:

- **append(eleman):** Listenin en sonuna yeni eleman ekler.
- **insert(indeks, eleman):** Belirtilen indekse eleman sıkıştırır.
- **pop(indeks):** Belirtilen indeksteki elemanı çıkarır ve döndürür.
- **remove(değer):** İsmi verilen ilk elemanı listeden siler.

PYTHON KODU

```
sehirler = ["İstanbul", "Ankara"]

sehirler.append("İzmir")          # Sona ekler -> ["İstanbul", "Ankara", "İzmir"]
sehirler.insert(1, "Bursa")       # 1. indekse ekler -> ["İstanbul", "Bursa", "Ankara",
silinen = sehirler.pop(2)         "İzmir"]
sehirler.remove("İstanbul")       # 2. indeksteki "Ankara"yı çıkarır
                                  # "İstanbul"u siler

print("Çıkarılan Şehir:", silinen)
print("Listenin Son Hali:", sehirler)
```

```
# Ekran Çıktısı:
Çıkarılan Şehir: Ankara
Listenin Son Hali: ['Bursa', 'İzmir']
```

GÖREV SLAYTI

ŞİMDİ SEN KODLA 2 – Alışveriş Sepeti Uygulaması

GÖREV: Dinamik Alışveriş Sepeti Yönetimi

- 1. Boş bir liste oluştur: sepet = [ ]
- 2. append() metodunu kullanarak sepete sırasıyla **'Elma'**, **'Süt'** ve **'Ekmek'** ekle. • 3. remove() metodu ile sepetten **'Süt'** ürününü çıkar.
- 4. Sepetin son halini ve sepetteki ürün sayısını ekrana yazdır.

PYTHON KODU

```
# SENİN KODLAYACAĞIN
ŞABLON: sepet = []

# 1. Ürünleri append ile ekle


# 2. 'Süt' ürününü remove ile çıkar


# 3. Sepet durumunu ve len() değerini yazdır
```

Ekran Çıktısı:

Kodunu çalıştırdığında sepette 2 ürün kalmalı ve ['Elma', 'Ekmek'] listelenmeli!)

ÇÖZÜM SLAYTI

ŞİMDİ SEN KODLA 2 – Tam Çözüm

ÇÖZÜM KODU İNCELEMESİ:

• sepet = [] ile boş liste tanımlandı.

• Üç adet append() çağrısı ile elemanlar sona eklendi. • remove('Süt') çağrısı ile 'Süt' listeden kaldırıldı.

PYTHON KODU

```
# TAM ÇALIŞAN ÇÖZÜM KODU sepet = []
# 3 Ürün Ekleme
sepet.append("Elma")
sepet.append("Süt")
sepet.append("Ekmek")
print("Ürün Eklendi:", sepet)

# 1 Ürün Çıkarma
sepet.remove("Süt")
print("Süt Çıkarıldı:", sepet)

# Sepet Özeti
print("Kalan Ürün Sayısı:", len(sepet))
print("Son Sepet İçeriği:", sepet)
```

```
# Ekran Çıktısı:
Ürün Eklendi: ['Elma', 'Süt', 'Ekmek']
Süt Çıkarıldı: ['Elma', 'Ekmek']
Kalan Ürün Sayısı: 2
Son Sepet İçeriği: ['Elma', 'Ekmek']
```

Döngü (for) ve Koşul (if/else) ile Liste İşleme

Liste elemanları üzerinde for döngüsü ile tek tek gezilebilir. Döngü içinde if/else kullanılarak filtrelenmiş işlemler yapılır.

PYTHON KODU

```
notlar = [45, 78, 90, 32, 65, 50] gecenler = []
kalanlar = []

for puan in notlar:
        if puan >= 50:
            gecenler.append(puan)
        else:
            kalanlar.append(puan)

print("Tüm Notlar:", notlar) print("Geçen Notlar (>=50):", gecenler)
print("Kalan Notlar (<50):", kalanlar)
```

```
# Ekran Çıktısı:
Tüm Notlar: [45, 78, 90, 32, 65, 50]
Geçen Notlar (>=50): [78, 90, 65, 50]
Kalan Notlar (<50): [45, 32]
```

GÖREV SLAYTI

ŞİMDİ SEN KODLA 3 – Fiyat İndirimi Hesaplayıcı

GÖREV: 100 TL Üstü Ürünlere İndirim Uygulama

- Verilen fiyat listesi: fiyatlar = [120, 80, 250, 45, 150]
- 1. Bir for döngüsü kurarak her fiyatı incele.
- 2. Eğer fiyat **100 TL veya üstündeyse** %20 indirim uygula (fiyat * 0.80).
- 3. 100 TL altındaki fiyatları değiştirmeden koru.
- 4. Oluşan yeni fiyatları yeni_fiyatlar adında yeni bir listeye ekleyip yazdır.

PYTHON KODU

```
fiyatlar = [120, 80, 250, 45, 150]
yeni_fiyatlar = []

# Döngü ve if/else yapını yaz:
# 100 TL üstü ise %20 indirimli ekle, değilse aynen ekle
```

Ekran Çıktısı:

Beklenen Çıktı: Yeni Fiyatlar: [96.0, 80, 200.0, 45, 120.0])

ÇÖZÜM SLAYTI

ŞİMDİ SEN KODLA 3 – Tam Çözüm

ÇÖZÜM KODU İNCELEMESİ:

- for f in fiyatlar: ile tüm elemanlar tek tek işlendi.
- if f >= 100: şartı sağlayan elemanlara indirim uygulandı.
- else: ile 100 TL altı fiyatlar direkt yeni listeye eklendi.

PYTHON KODU

```
# TAM ÇALIŞAN ÇÖZÜM KODU
fiyatlar = [120, 80, 250, 45, 150]
yeni_fiyatlar = []

for f in fiyatlar:
    if f >= 100:
        indirimli = f * 0.80
        yeni_fiyatlar.append(indirimli)
    else:
        yeni_fiyatlar.append(f)

print("Orijinal Fiyatlar:", fiyatlar)
print("İndirimli Fiyatlar:", yeni_fiyatlar)
```

```
# Ekran Çıktısı:
Orijinal Fiyatlar: [120, 80, 250, 45, 150]
İndirimli Fiyatlar: [96.0, 80, 200.0, 45, 120.0]
```

Sözlükler (Dictionaries) Nedir?

Sözlükler{anahtar:değer}(key-value)çiftlerihalindeveri tutar. İndeks numarası yerine anlamlı **anahtar (key)** isimleri kullanılır.

PYTHON KODU

```
# Öğrenci Profil Kartı Sözlüğü
ogrenci = {
    "ad": "Ahmet",
    "soyad": "Yılmaz",
    "yas": 16,
    "not_ortalamasi": 85.5,
    "aktif": True
}

print("Öğrenci Adı:", ogrenci["ad"])
print("Öğrenci Yaşı:", ogrenci["yas"])
print("Tüm Sözlük:", ogrenci)
```

```
# Ekran Çıktısı:
Öğrenci Adı: Ahmet
Öğrenci Yaşı: 16
Tüm Sözlük: {'ad': 'Ahmet', 'soyad': 'Yılmaz', 'yas': 16, 'not_ortalamasi': 85.5, 'aktif': True}
```

Sözlük İşlemleri ve Metotları

Sözlüklerdeveriokuma,güvenlierişim(.get()),ekleme, güncelleme ve .items() döngüsü:

PYTHON KODU

```
urun = {"ad": "Laptop", "fiyat": 15000}

# 1. Güvenli Erişim (.get) - Olmayan anahtar hata vermez, None döner
print("Stok:", urun.get("stok", "Bilgi Yok"))


# 2. Veri Ekleme ve Güncelleme
urun["stok"] = 50                    # Yeni anahtar ekler
urun["fiyat"] = 16500                # Var olanı günceller


# 3. .items() ile Anahtar-Değer Döngüsü for anahtar,
deger in urun.items(): print(anahtar, "->", deger)
```

Ekran Çıktısı:
Stok: Bilgi Yok ad -> Laptop fiyat -> 16500 stok -> 50

GÖREV SLAYTI

ŞİMDİ SEN KODLA 4 – Oyun Karakteri Kartı

GÖREV: Oyun Karakteri Durum Güncellemesi

- 1. Bir sözlük tanımla: karakter = {'ad': 'Savaşçı', 'seviye': 5, 'can': 100}
- 2. Karakter 35 puan hasar alsın. can değerini 35 azaltarak güncelle.
- 3. Karakterin seviyesini 1 artır (6 yap).
- 4. for k, v in karakter.items(): döngüsü ile karakterin son özelliklerini ekrana yazdır.

PYTHON KODU

```
karakter = {"ad": "Savaşçı", "seviye": 5, "can": 100}

# 1. Hasar al (canı 35 eksilt):

# 2. Seviye atla (seviyeyi 1 artır):

# 3. .items() döngüsü ile yazdır::
```

Ekran Çıktısı:
Beklenen Çıktı: can -> 65, seviye -> 6 olarak güncellenmiş görünmeli!)

ÇÖZÜM SLAYTI

ŞİMDİ SEN KODLA 4 – Tam Çözüm

ÇÖZÜM KODU İNCELEMESİ:
• karakter['can'] -= 35 ile sözlükteki sayısal değer güncellendi.
• karakter['seviye'] += 1 ile seviye artırıldı.
• .items() metodu anahtar ve değer çiftlerini aynı anda döngüye sundu.

PYTHON KODU

```
# TAM ÇALIŞAN ÇÖZÜM KODU
karakter = {"ad": "Savaşçı", "seviye": 5, "can": 100}


 # Hasar alma ve seviye atlama
 karakter["can"] -= 35
 karakter["seviye"] += 1

 print("--- KARAKTER DURUM KARTI ---")
 for ozellik, deger in karakter.items():
         print(ozellik.upper(), ":", deger)
```

Ekran Çıktısı:

```
--- KARAKTER DURUM KARTI ---
AD : Savaşçı
SEVIYE : 6
CAN : 65
```

Gerçek Dünya Yapısı: Liste İçinde Sözlükler

Gerçekyazılımlardaveveritabanlarındaverilergenellikle**sözlüklerdenoluşan listeler** şeklinde tutulur. Her sözlük bir kaydı temsil eder.

PYTHON KODU

```
ogrenciler = [
    {"ad": "Ali", "not": 70},
    {"ad": "Ayşe", "not": 95},
    {"ad": "Mehmet", "not": 40}
]


for ogrenci in ogrenciler:
    isim = ogrenci["ad"]
    puan = ogrenci["not"]
    if puan >= 50:
        durum = "GEÇTİ"
    else:
        durum = "KALDI"
    print(f"Öğrenci: {isim} | Not: {puan} | Durum: {durum}")
```

Ekran Çıktısı:

Öğrenci: Ali | Not: 70 | Durum: GEÇTİ
Öğrenci: Ayşe | Not: 95 | Durum:
GEÇTİ Öğrenci: Mehmet | Not: 40 |
Durum: KALDI

Liste mi, Sözlük mü? Günün Özeti ve Karşılaştırma

Özellik	Liste (List)	Sözlük (Dictionary)
Sözdizimi	[1,2, 3]	{'a':1,'b': 2}
Erişim Mantığı	Sıralı İndeks (0, 1, 2...)	Anahtar (Key) Adı ile
En Uygun Kullanım	Aynı türdeki veri dizileri (Notlar, Ürünler, Şehirler)	Tek bir varlığın detaylı özellikleri (Öğrenci Profili, Ürün Kartı)
Metotlar	append(), pop(), remove()	get(), keys(), values(), items()

4. GÜN ÖZETİ: Bugün Python'da çoklu verileri depolamayı, listeler ile indeks tabanlı erisimi, sözlükler ile anahtar-deger ilişkilerini ve döngülerle veri işleme pratiklerini öğrendik!

ATÖLYE GÖREVİ

BÜYÜK ATÖLYE – Mini Sipariş Takip Sistemi

ATÖLYE GÖREVİ: Döngü, Koşul, Liste ve Sözlük Bütünleşmesi

- Tüm öğrendiklerimizi birleştiriyoruz!
- 1. Sipariş listesi oluştur: siparisler = [{'urun': 'Kulaklık', 'fiyat': 350}, {'urun': 'Klavye', 'fiyat': 600}]
- 2. yeni_urun ve yeni_fiyat değişkenleri tanımlayıp append() ile siparişlere 3. ürünü ekle.
- 3. Bir while veya for döngüsü kurarak toplam sipariş tutarını hesapla.
- 4. Toplam tutar 1000 TL üstündeyse **'Kargo Bedava!'**, değilse **'50 TL Kargo Ücreti Ekleyin'** yazdır.

PYTHON KODU

```
siparisler = [
    {"urun": "Kulaklık", "fiyat": 350},
    {"urun": "Klavye", "fiyat": 600}
]

# 1. Yeni sipariş ekle (Fare, 200 TL)
# 2. Döngü ile toplam fiyatı hesapla
# 3. Kargo durumunu kontrol et
```

Ekran Çıktısı:
Beklenen Çıktı:
3 Ürün, Toplam Tutar: 1150 TL -> Kargo Bedava!)

ATÖLYE ÇÖZÜMÜ

BÜYÜK ATÖLYE – Tam Çözüm ve Kod Analizi

SİSTEM KODU İNCELEMESİ:
• Yeni sipariş sözlük olarak append() ile sepete eklendi.
• for s in siparisler: döngüsü her bir ürüne ulaşıp fiyatı toplam değişkeninde biriktirdi.
• if/else bloku kargo avantajını kontrol etti.

PYTHON KODU

```
siparisler = [
    {"urun": "Kulaklık", "fiyat": 350},
    {"urun": "Klavye", "fiyat": 600}
]

# Yeni Ürün Ekleme
yeni = {"urun": "Fare", "fiyat": 200}
siparisler.append(yeni)

toplam = 0
for s in siparisler:
    print(f"Ürün: {s['urun']} - Fiyat: {s['fiyat']} TL")
    toplam += s["fiyat"]

print("----------------------------------")
print("Toplam Sipariş Tutarı:", toplam, "TL")
if toplam >= 1000:
    print("Kargo Durumu: KARGO BEDAVA!")
else:
    print("Kargo Durumu: 50 TL Kargo Ücreti Eklendi.")
```

Ekran Çıktısı:

```
Ürün: Kulaklık - Fiyat: 350 TL
Ürün: Klavye - Fiyat: 600 TL
Ürün: Fare - Fiyat: 200 TL
---------------------------------
Toplam Sipariş Tutarı: 1150 TL
Kargo Durumu: KARGO BEDAVA!
```